

CHAPTER - 4

ANALYZING FUNCTIONAL DEPENDENCIES, REDUNDANCY AND DATA ANOMALIES

4.1 INTRODUCTION

This chapter examines the internal design of the Sephora Beauty E-Commerce Order Management Database at the level of individual tables — customers, categories, products, orders, order items, payments, shipments, and administrators. For each table, the functional dependencies are identified first, since they reveal exactly which attributes are determined by the primary key. That analysis is then used to check the table against redundancy and the three classical data anomalies — insert, update, and delete — so that the reasoning behind the schema's normalization is made explicit rather than assumed.

4.2 METHODOLOGY

Each table below is discussed under three headings. Functional Dependency states the determinant (the primary key) and the attributes it fully determines, along with a brief note on how foreign keys connect that table to the rest of the schema. Redundancy Analysis explains, in practical terms, why the attribute is stored only once and references the level of normal form the table satisfies. Data Anomalies then works through the insert, update, and delete cases individually, showing why the current structure avoids each one rather than simply asserting that it does.

1. CUSTOMER TABLE

Functional Dependency

Customer_ID → Customer_Name, Email, Phone_Number, Address

Customer_ID is the primary key of this table, and it is the only attribute that every other order in the system uses to reach a customer's details.

Redundancy Analysis

Because every non-key attribute here depends solely on Customer_ID, a customer's name, email, phone number, and address are written down exactly once, no matter how many orders that person eventually places.

The table satisfies third normal form (3NF): there is no attribute here that depends on another non-key attribute rather than on Customer_ID directly.

Data Anomalies

- A customer can be registered in the system the moment they sign up, well before they place a first order — so there is no insert anomaly tying customer creation to order creation.
- If a customer's phone number changes, exactly one row in one table needs editing, which rules out the update anomaly of having to chase down duplicate copies.
- Cancelling or deleting every order a customer has ever placed still leaves that customer's profile in the system, so no delete anomaly wipes out customer data by accident.

2. CATEGORY TABLE

Functional Dependency

Category_ID → Category_Name

Category_ID uniquely identifies a category, and Product rows link to it through a foreign key instead of storing the category text directly.

Redundancy Analysis

A category's name exists in a single row of this table; every product that belongs to it — skincare, makeup, haircare, or fragrance — simply carries the matching Category_ID, so the name is never typed out more than once across the database.

With only one non-key attribute depending entirely on the key, the Category table trivially meets 2NF and 3NF requirements.

Data Anomalies

- Brand-new categories — a seasonal makeup line, for instance — can be added to the catalogue structure before a single product is placed under them.
- Should a category ever need renaming, that edit happens in one place and is reflected everywhere the Category_ID is referenced.
- Discontinuing and removing the last product under a category leaves the category definition itself untouched.

3. PRODUCT TABLE

Functional Dependency

Product_ID → Product_Name, Price, Stock, Category_ID

Product_ID is the determinant for this table; every attribute listed — name, price, stock count, and category — is fully dependent on it and nothing else, whether the product is a foundation, a serum, or a perfume.

Redundancy Analysis

Order-related tables never repeat a product's name or price; they simply carry the Product_ID and look up the details from this single source of truth, which matters for a beauty catalogue where prices and stock change frequently across seasonal collections.

Since Category_ID (a foreign key) does not determine Price or Stock, there is no transitive dependency, which keeps the table in 3NF.

Data Anomalies

- New stock can be entered into the catalogue in advance of any customer ordering it, avoiding an insert anomaly.
- A price change for a product is a one-row update in the Product table, rather than a search-and-replace across every past order.
- Removing a historical order does not touch the product catalogue, so the product and its stock count remain exactly as they were.

4. ORDERS TABLE

Functional Dependency

Order_ID → Customer_ID, Order_Date, Total_Amount, Order_Status

Order_ID is the primary key, while Customer_ID appears here only as a foreign key pointing back to the Customer table.

Redundancy Analysis

Only order-level facts — who placed it, when, for how much, and its current status — live in this table; the customer's own details are never copied into it.

Because Total_Amount and Order_Status describe the order itself rather than the customer, this table avoids the repeating-group and transitive-dependency problems that 2NF and 3NF guard against.

Data Anomalies

- As long as a valid Customer_ID exists, a new order can be logged without needing any product or payment information to already be in place.
- Marking an order as shipped or cancelled changes only the Order_Status column, leaving every other table untouched.
- Deleting an order record does not cascade into deleting the customer who placed it or the products that were ordered.

5. ORDER_ITEM TABLE

Functional Dependency

Order_Item_ID $\rightarrow$ Order_ID, Product_ID, Quantity, Price

This table resolves the many-to-many relationship between Orders and Products, using Order_Item_ID as a surrogate key alongside the Order_ID and Product_ID foreign keys.

Redundancy Analysis

Rather than listing every purchased product inside the Orders row itself, each product-and-quantity pairing gets its own Order_Item row, keeping the Orders table free of repeating groups.

This decomposition is exactly what separates the design from an unnormalized 1NF violation, where a single order row would otherwise need a variable number of product columns.

Data Anomalies

- A product can be added as a line item to an order without needing to modify any other order or any other line item.
- Adjusting the quantity on one line item has no bearing on quantities recorded for other items, even within the same order.
- Deleting a single line item removes only that product's association with that order — the product itself stays in the Product table.

6. PAYMENT TABLE

Functional Dependency

Payment_ID $\rightarrow$ Order_ID, Payment_Method, Payment_Status

Payment_ID identifies each transaction, with Order_ID stored as a foreign key rather than repeating any order details inside the Payment table.

Redundancy Analysis

Keeping payment method and payment status in their own table means that if a customer retries a failed payment, the system records a new Payment row without altering the Orders table at all.

Payment_Status depends only on Payment_ID and not on Order_ID or any other attribute, so no partial or transitive dependency is introduced.

Data Anomalies

- A payment attempt can only be logged once its associated order already exists, which is by design rather than a flaw.
- Flipping a payment from 'Pending' to 'Completed' is a single-column update confined to the Payment table.
- Purging an old or failed payment record has no effect on the order it was attached to.

7. SHIPMENT TABLE

Functional Dependency

Shipment_ID → Order_ID, Tracking_Number, Delivery_Status

Shipment_ID is the key here, and Order_ID is carried purely as a reference back to the order being fulfilled.

Redundancy Analysis

Tracking numbers and delivery statuses are recorded once per shipment, separate from the order data, which means a single order that ships in multiple parcels doesn't force duplicate order rows. Delivery_Status is functionally dependent on Shipment_ID alone, so the table introduces no redundancy back into the Orders table.

Data Anomalies

- A shipment entry only gets created once an order has actually been confirmed for dispatch, which is expected rather than problematic.
- Updating a parcel's delivery status — 'In Transit' to 'Delivered', for example — touches only the Shipment table.
- Clearing out an old shipment record after delivery has no impact on the order history it was tied to.

8. ADMIN TABLE

Functional Dependency

Admin_ID → Admin_Name, Email, Password

Admin_ID uniquely identifies each administrator account, entirely independent of the customer- and order-related tables.

Redundancy Analysis

Login credentials for staff are isolated in their own table, so no other part of the schema ever needs to store or repeat an administrator's password or email.

Every attribute in this table depends only on Admin_ID, satisfying 3NF and keeping administrative data cleanly separated from operational data.

Data Anomalies

- A new administrator account can be provisioned without any dependency on existing customer or order records.
- Updating an admin's email or password is confined to a single row in the Admin table.
- Revoking or deleting an administrator's access leaves every customer and order record completely unaffected.

CONCLUSION

Taken together, the eight tables that make up the Sephora Beauty E-Commerce database each depend on a single, well-chosen primary key, and no non-key attribute in any of them depends on anything other than that key. This is what keeps the schema in third normal form throughout: customer, product, and administrator information is never copied across tables, and the linking tables — Orders, Order_Item, Payment, and Shipment — carry only foreign keys rather than duplicated data. The practical payoff is that inserting, updating, or deleting a record in one table does not create unwanted side effects elsewhere, which is precisely the guarantee a normalized schema is meant to provide for a system handling continuous order activity across a beauty and cosmetics catalogue.